



Instrucciones generales del parcial

- **Debes usar exclusivamente** las implementaciones base de Pila y Cola que te proveo.
- Todas tus funciones deben incluir **TypeHints**.
- Para la **Cola de Prioridad**, si el ejercicio la requiere, debes programarla tú (puedes implementarla con listas de Python).
- **Prohibido** usar cualquier otra colección (listas, sets, diccionarios, arrays dinámicos, etc.) **como estructura de solución para los ejercicios del parcial**.
 - Nota: En algunos ejercicios **se recibirán strings o listas como parámetros**, pero **toda la lógica** debe basarse **solo** en **pila, cola o cola de prioridad**, según el enunciado.
- Si modificas una estructura durante el proceso, **debes dejarla en su estado original** al finalizar (salvo que el enunciado diga lo contrario).
- Escribe **funciones puras** y **bien documentadas**; evita Entrada/Salida en consola salvo para pruebas.
- Todos los parciales están sujetos a sustentación. Si se detecta indicios de fraude, será citado a sustentación.
- Cualquier intento de fraude anulará el parcial.

1) Pila vs Cola: ¿mismos valores y mismo orden?

Objetivo. Implementa una función que determine si **una cola** y **una pila** contienen **exactamente los mismos valores en el mismo orden lógico**.

Definición de “mismo orden lógico”:

- Para la **cola**, el orden es **frente** → **fondo**.
- Para la **pila**, el orden es **base** → **tope**.

Es decir, si lees la cola de frente a fondo y la pila de base a tope, deben producir la **misma secuencia**.

Restricciones.

- Usa **solo** las operaciones permitidas por las implementaciones base.
- **Debes restaurar** la pila y la cola a su **estado original**.
- **No** puedes copiar elementos a estructuras no permitidas.

Ejemplo (conceptual).

- Cola (frente→fondo): 1, 3, 5
- Pila (base→tope): 1, 3, 5 → **True**
- Nota: Si la pila es (tope hacia arriba) 5, 3, 1, su **base→tope** es 1, 3, 5.

2) ¿Es palíndromo? (puedes usar pila y cola)

Objetivo. Implementa una función que determine si un **string** es **palíndromo**.

Normalización (obligatoria).

- Ignora **espacios**, **tildes** y **signos de puntuación**.
- No distingas entre **mayúsculas/minúsculas**.

Restricciones.

- Puedes usar **pila** y **cola** (ambas o una, como prefieras).
- Toda la verificación debe basarse en estas estructuras.
- Nada de slicing ni estructuras prohibidas para comparar.

Ejemplos.

- "Anita lava la tina" → True
- "A man, a plan, a canal: Panama!" → True
- "Hola" → False

3) 🚒 Centro de emergencias: cola de prioridad por tipo de evento

Contexto. Eres el sistema de atención de emergencias de una ciudad. Debes **atender eventos** con esta **prioridad (de mayor a menor)**:

1. **Incendio** 🔥
2. **Accidente** 🚗
3. **Consulta** 🩹

Reglas de atención.

- Atiende **siempre** primero los **incendios**; si no hay, los **accidentes**; si no hay, las **consultas menores**.
- **Estabilidad (FIFO por prioridad)**: dentro de una misma prioridad, atiende **en orden de llegada**.

Restricciones.

- **Debes implementar una cola de prioridad** usando **listas de Python**.
- Nada de usar **heap** o colecciones no permitidas.

Entrada (sugerida).

- Una **lista** de eventos (solo como parámetro), donde cada evento es una tupla o registro con:
`tipo ∈ {"incendio", "accidente", "consulta"}, descripcion: str`

Salida.

- Devuelve la cola de prioridad (**como valor de retorno**) con el **orden de atención**.

Ejemplo (conceptual).

Entrada (llegadas):

1. (`"consulta"`, "Dolor leve")
2. (`"incendio"`, "Edificio A")
3. (`"accidente"`, "Choque simple")
4. (`"incendio"`, "Bodega X")
5. (`"consulta"`, "Ayyyy muchachos...")

Salida de atención:

- Incendio (Edificio A) → Incendio (Bodega X) → Accidente (Choque simple) → Consulta (Dolor Leve) → Consulta (Ayyyy muchachos...)